Hochschule Konstanz
Fakultät Informatik

Software Engineering

02 - Version Control System - Git

Prof. Dr. Marko Boger

Task 1.3 Learn Scala on Codetask

Codetask is an e-learning platform for Scala, developed at HTWG

https://codetask.in.htwg-konstanz.de

Try out Exercises and Art of Programming courses at your own speed and time.

Software Engineering is specifically for this lecture, but not quite complete yet.

Goals

In this lecture you will learn about

- Tuples, a data structure for values of different type
- Objects, a first encapsulation of data and functions with a single instance
- Enums, an abstraction with a fixed set of instances
- Classes, an abstraction with a variable set of instances

Supporting literature:

- Learn Scala 3 the Fast Way, chapter 20,67

New Tools:

- Cursor, Windsurf

About Types

A Type is the formal description of what form a parameter to a function can have, or a function can return, or a variable can have.

In JavaScript, we only have types Number, String and Boolean.

In Scala, simple examples are Int, Boolean, or String. But also Array and List.

```
val x: Int = 42;
var name: String = "Luke";
def f(x: Int): Int = x + 1
def sum(xs: List[Int]): Int = {...}
```

But also mappings between types:

```
def f1(x: Int): Int => Int = x => x + 1
def f2(x: Int): Int = x + 2
def f3(x: Int) = x + 3
def isEven(x: Int): Int => Boolean = x => x % 2 == 0
def isOdd(x: Int): Boolean = x % 2 != 0
```

In this lecture, we will get to know more Types.

Tuples

We have seen Array and List, which hold data of the same type.

But what if we have data of different type?

A simple solution is the Tuple type.

```
val tuple = (42, "Luke", true)// :Tuple3[Int, String, Boolean]
tuple._1 // : 42
tuple._2 // : "Luke"
tuple._3 // : true
```

Scala has Tuple2, Tuple3, Tuple4, … , Tuple22, …

Tuple are immutable.

Their fields can be accessed with _1, _2, …

or using Tuple(0), Tuple(1) …

Tuples compared to Struct

Other languages have similar data types, especially procedural languages, like C.

C

```c
struct Person {
    char name[50];
    int age;
};
struct Person p = {"Luke", 42};
p.age = 67; // Mutable
```

snappify.com

In C, Structs are mutable, and fields are named.

In Scala, Tuple are immutable, and fields are anonymous.

Usage of Tuple

Tuples are usually not used to model important data structures. They are used in a light-weight fashion, for example to return two results from a function.

```
def div(x: Int, y: Int): (Int, Int) = (x / y, x % y)

val divResult = div(10, 3) // : Tuple2[Int, Int] = (3,1)
divResult._1 // : Int = 3
divResult._2 // : Int = 1
```

Assignment to Tuple

A Tuple can be assigned to another Tuple. In this step, the anonymous fields can be assigned to named fields.

```
val (quotient, remainder) = div(10, 3)
quotient // : Int = 3
remainder // : Int = 1
```

This is also referred to as pattern matching.

However, do not over-use Tuples, for important data structures we have more tools.

Object

Objects are much like the sprites from Scratch. They can have data and functions. But there is only one instance of them. This is sometimes called a singleton.

```
object Earth {
 // Data
 val radius: Double = 6371.0 // Average Earth radius in km
 val pi: Double = 3.14159
 var population: Int = 8_000_000_000

 // Function: Returns surface area and equatorial circumference
 def surfaceArea(): Double = 4 * pi * radius * radius
 def circumference(): Double = 2 * pi * radius
}

val area = Earth.surfaceArea()
val circumference = Earth.circumference()
println(s"Earth: Surface Area = $area km², Equatorial
Circumference = $circumference km")
```

Addressing Objects

Objects can be accessed directly by their name. We do not have to pass a reference.

This has some advantages. For example for constances, utilities, of factories. Objects are often used as the starting point of an application.

```
object MyFirstApplication {
 def main(args: Array[String]): Unit = {
   println("Hello, World!")
 }
}
// instead of an object we can use @main
@main def hello(): Unit = println("Hello, World!")
```

But it can also lead to messy code. So use them carefully, and sparsely.

Enums

Enums can model data structures of which there is a small and fixed amount, like Weekdays or CardTypes. The instances can just be separated by a comma or by a case keyword.

```
enum Weekday {
 case Monday, Tuesday, Wednesday, Thursday, Friday,
   Saturday, Sunday
}

enum CardType {
 case Heart, Diamond
 case Club, Spade
}
```

Console Colors

A nice example for an Enum are the colors of the console:

```
enum ConsoleColors(val code: String) {
 case CLEAR extends ConsoleColors("\u001B[0m")
 case RED extends ConsoleColors("\u001B[31m")
 case GREEN extends ConsoleColors("\u001B[32m")
 case YELLOW extends ConsoleColors("\u001B[33m")
 case BLUE extends ConsoleColors("\u001B[34m")
 case PURPLE extends ConsoleColors("\u001B[35m")
 case CYAN extends ConsoleColors("\u001B[36m")
 case WHITE extends ConsoleColors("\u001B[37m")

 def apply(text: String): String = s"$code$text${CLEAR.code}"
}
```

Importing Enums

Enums, like Objects, can be addressed by their full name

```
ConsoleColors.RED("Error!")
```

But if you import the Enum, you can use the cases directly

```
import ConsoleColors._
RED("Error!")
YELLOW("Warning!")
GREEN("Success!")
```

Then it can very nicely be used for colored output to the console

```
println(RED("Error!"))
```

Classes

Classes are similar to Objects or Enums. But while Objects and Enums are directly accessible, classes are only templates to create an object. They can not be accessed directly, we need to instantiate them.

We use them for data structures of which we have a varying number and potentially a lot of them.

Classes usually are named after subjects of a domain, like Person or Planet.

```
class Person(name: String, age: Int)
class Planet(name: String, radius: Double)
```

Instantiating Objects from Classes

The Class is a template from which objects are instantiated.

```
class Person(name: String, age: Int)
class Planet(name: String, radius: Double)
```

To create a new instance, the new keyword can be used

```
val luke = new Person("Luke", 42)
val earth = new Planet("Earth", 6371.0)
```

In many situations, the new keyword can be omitted

```
val lea = Person("Lea", 25)
val mars = Planet("Mars", 3390.0)
```

Note: Classes are written with a capital first letter, so are objects. But instances of classes are assigned to variables/values and they are written with a small first letter.

Classes can have Data and Functions

Classes can have their internal data and their own functions. Functions of a class are called Methods. Methods can be called on the instance.

```
class Planet(name: String, radius: Double) {
 val pi: Double = 3.14159
 def surfaceArea(): Double = 4 * pi * radius * radius
 def circumference(): Double = 2 * pi * radius
}

val earth = new Planet("Earth", 6371.0)
earth.surfaceArea()
earth.circumference()
```

Parameters of Classes

The Parameters of a Class are part of their internal data. They are initiated at the creation of the creation of the instance. They can be passed in the order of their appearance or they can be called by their name.

```
val luke = new Person("Luke", 42)
val lea = new Person(name = "Lea", age = 25)
val anakin = new Person(age = 69, name = "Anakin")
```

Default Values for Parameters

Parameters can be assigned a default value. Then it is optional to provide this parameter at instantiation.

Rocky planets have a similar density, but gas planets have a very different density. So we can provide a default density and only provide a density for gas planets.

```
class Planet(name: String, radius: Double, density: Double = 5.514) {
    val pi: Double = 3.14159
    def surfaceArea(): Double = 4 * pi * radius * radius
    def circumference(): Double = 2 * pi * radius
    def mass(): Double = density * radius * radius * radius
}

val earth = new Planet("Earth", 6371.0)
val jupiter = new Planet("Jupiter", radius = 71492.0, density = 1.326)
```

Evaluation at Construction

Values are calculate at the creation time of an instance. So, in case of the Planet, circumference, mass and volume can be calculated just once at construction.

```
class Planet(name: String, radius: Double, density: Double = 5.514) {
   val pi: Double = 3.14159
   val surfaceArea: Double = 4 * pi * radius * radius
   val circumference: Double = 2 * pi * radius
   val mass: Double = density * radius * radius * radius
}
```

Methods are calculated every time they are called.

Lazy Evaluation

So, methods are evaluated at every call. Values are evaluated at creation time. There is one more option. With a lazy evaluation, the value can be calculated once but at first call.

So for the planet we can use this.

```
class Planet(name: String, radius: Double, density: Double = 5.514) {
  val pi: Double = 3.14159
  lazy val surfaceArea: Double = 4 * pi * radius * radius
  lazy val circumference: Double = 2 * pi * radius
  lazy val mass: Double = density * radius * radius * radius
}
```

This is rarely used in normal classes. It will become relevant when we get to Streams and Modules.

Visibility

By default, methods of a class can be accessed from the outside.

But we can make these private. Now, pi and mass can not be accessed from outside.

Parameter are by default not visible, but if they are declared as var or val, they are.

```
 class Planet(val name: String, val radius: Double, density: Double =
5.514) {
   private val pi: Double = 3.14159
   def surfaceArea(): Double = 4 * pi * radius * radius
   def circumference(): Double = 2 * pi * radius
   private def mass(): Double = density * radius * radius * radius
 }
 println(earth.name)
 println(earth.radius)
 //println(earth.density) // does not compile
 println(earth.surfaceArea())
 println(earth.circumference())
 //println(earth.mass) // does not compile
```

Summary

Tuples, Objects, Enums and Classes all define **Types**. Like Int, String, Boolean, or Array and List.

Int, String, and Boolean are used for just one datum.

Arrays are used for a **fixed** number of data of the same type.

List are used for a **variable** number of data of the same type

Tuples are used for spontaneous data types of **mixed** data types.

Objects are used for a **single** instance of a data structure of mixed data types.

Enums are used for a **small and fixed** number of data structures.

Classes are used for a **variable number** of data structures.

Objects, Enums and Classes can have functions, then called **methods**.

Version Control

Almost no software project is done in isolation.

So we need collaboration mechanisms.

In software development different members of a team access shared resources.

This requires a coordination strategy and tooling:

- To synchronise access
- To avoid loss of changes

Version Control Systems

RCS
CVS
SVN / Subversion
Mercurial
Git

First Generation Local Repository

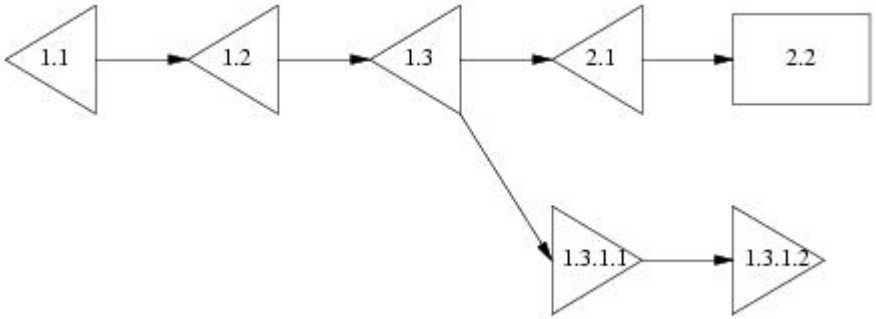
RCS or Revision Control System is a local version control system.

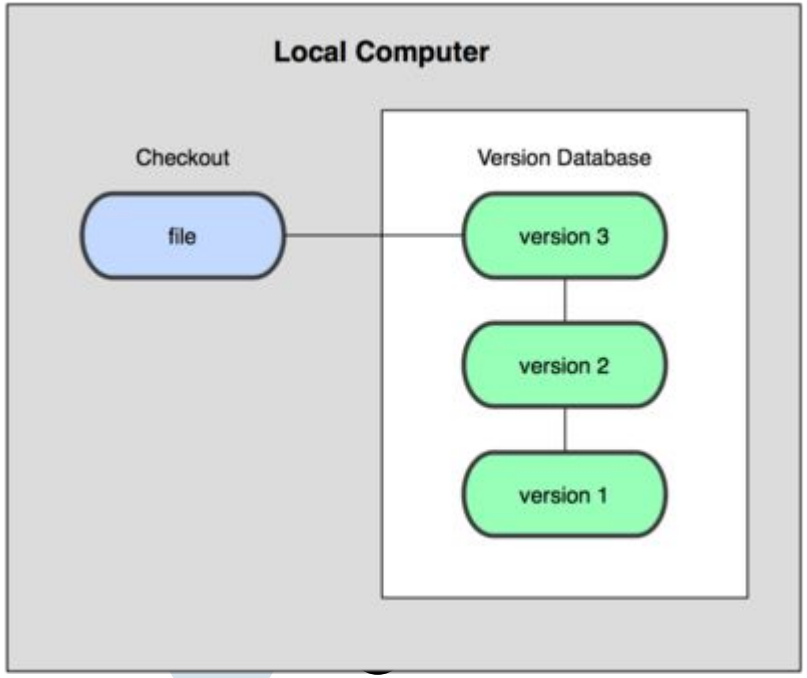
First released in 1982 by Walter Tichy.

It is file based and uses a locking strategy.

Version numbers are digits.

Storage is in deltas





Second Generation Centralized Repository

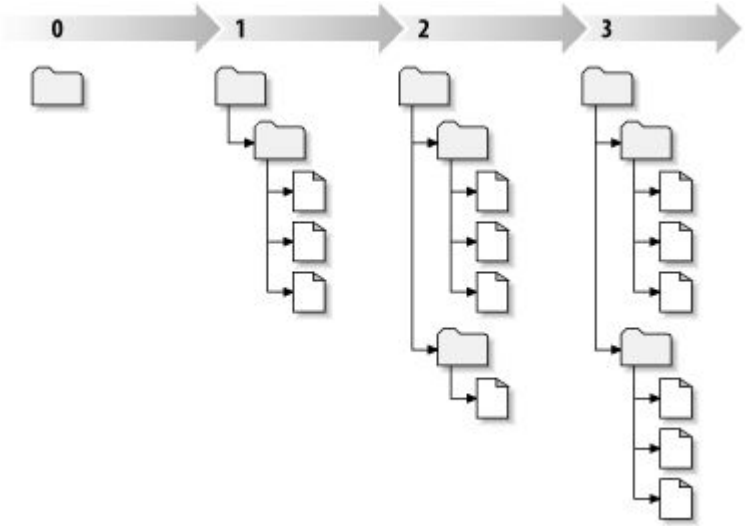
CVS or Concurrent Versions System is a centralized VCS.

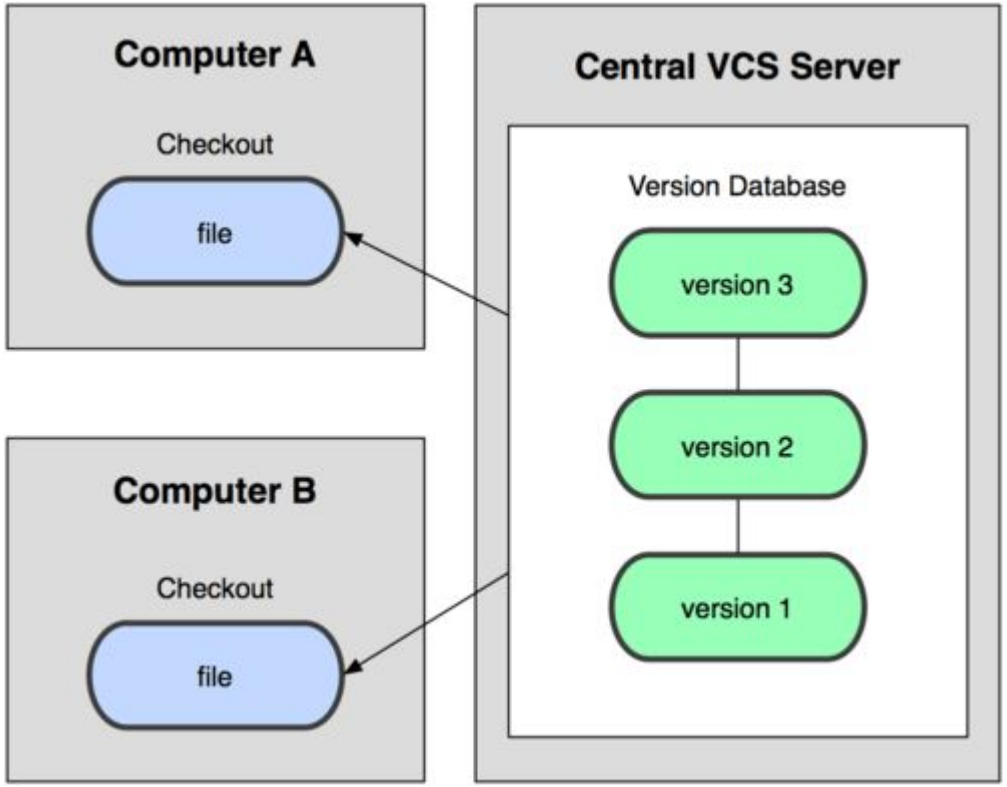
It was first released 1990.

SVN or Subversion was developed as a replacement for CVS.

SVN versions are digits

Versions go across a snapshot



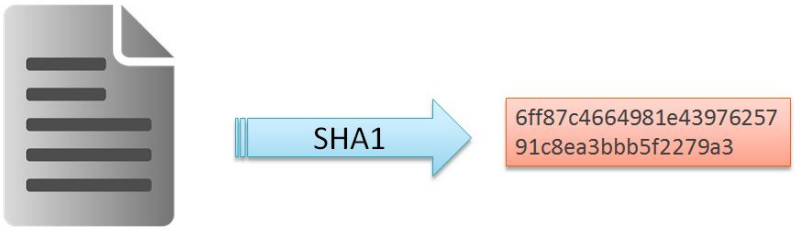


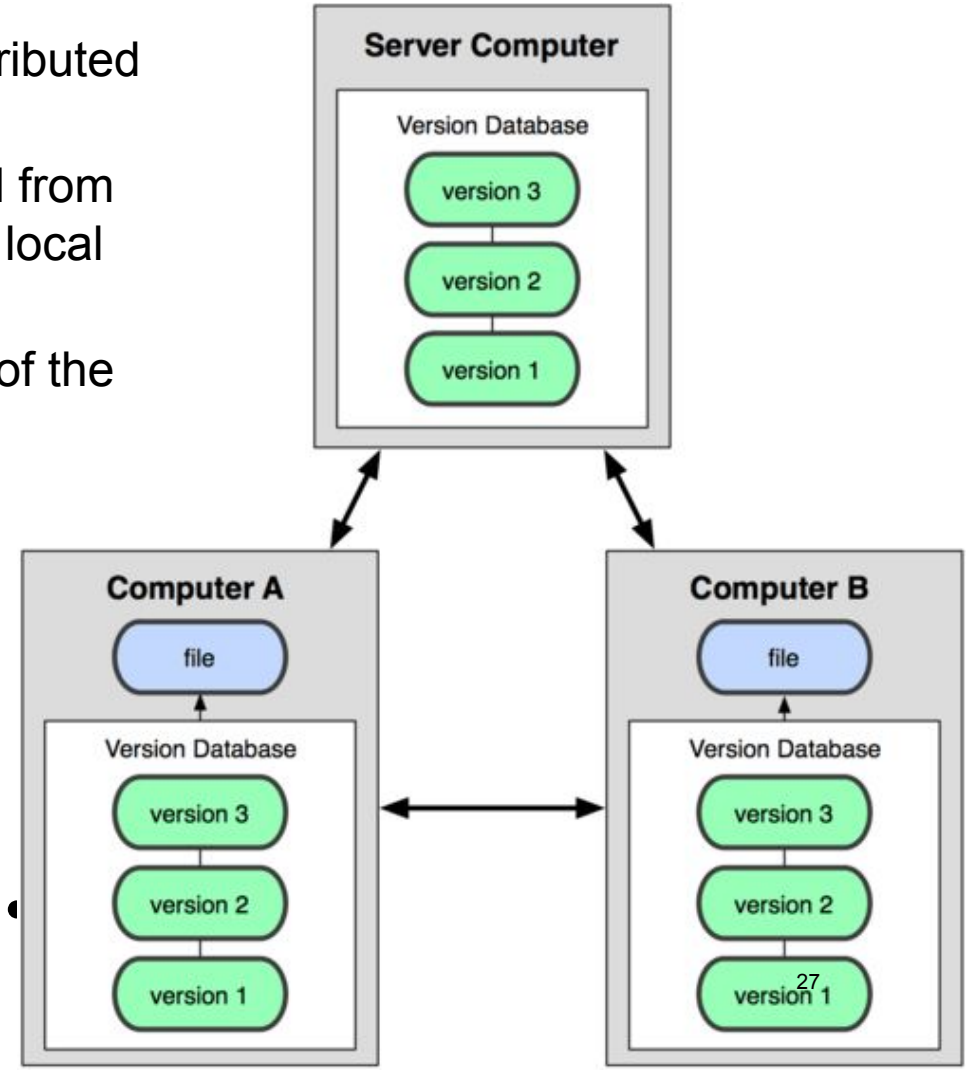
Third Generation Distributed Repository

Bit Keeper, Mercurial and git are distributed VCS.

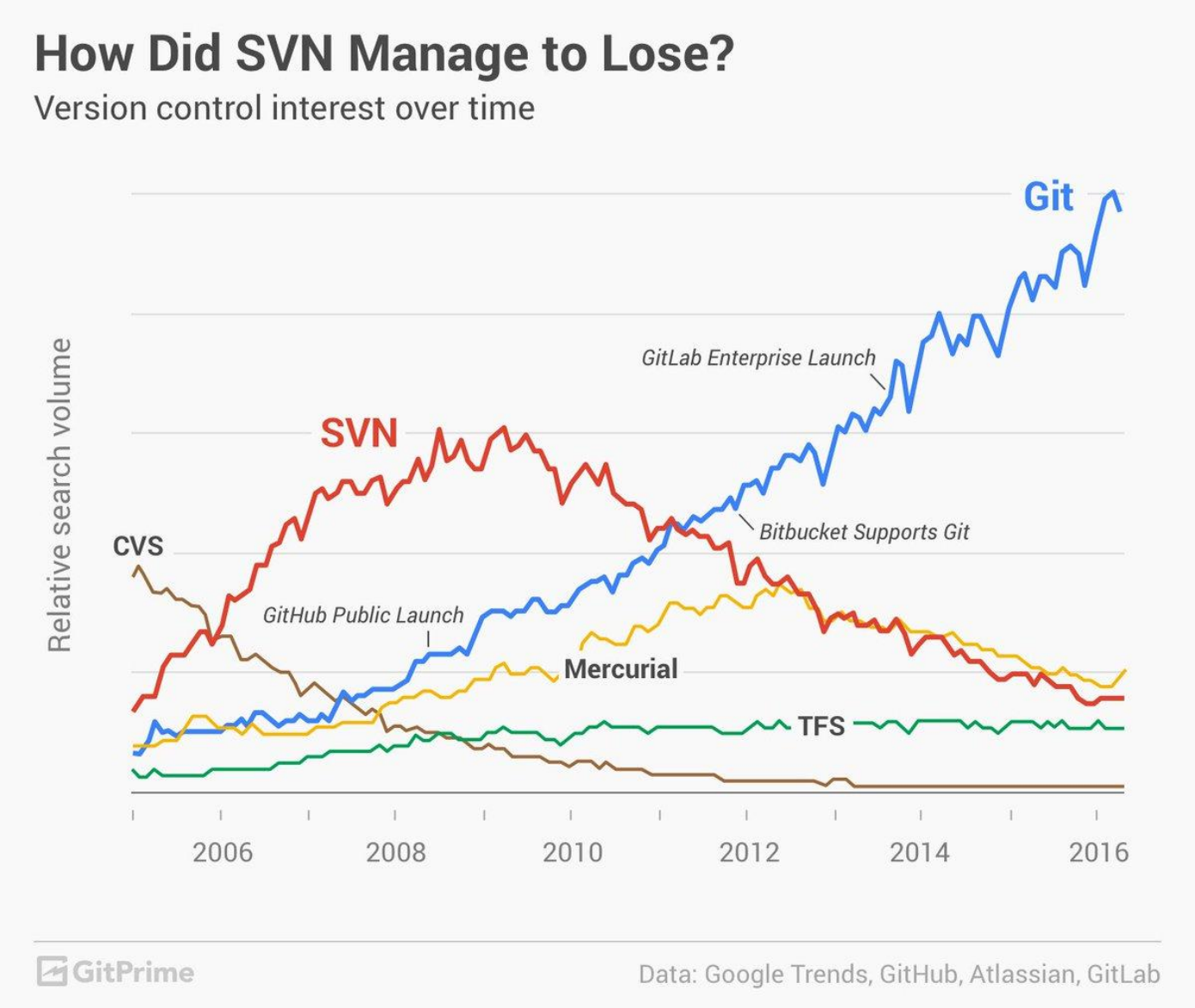
The complete repository is replicated from server to local computer. Server and local copy become equivalent.

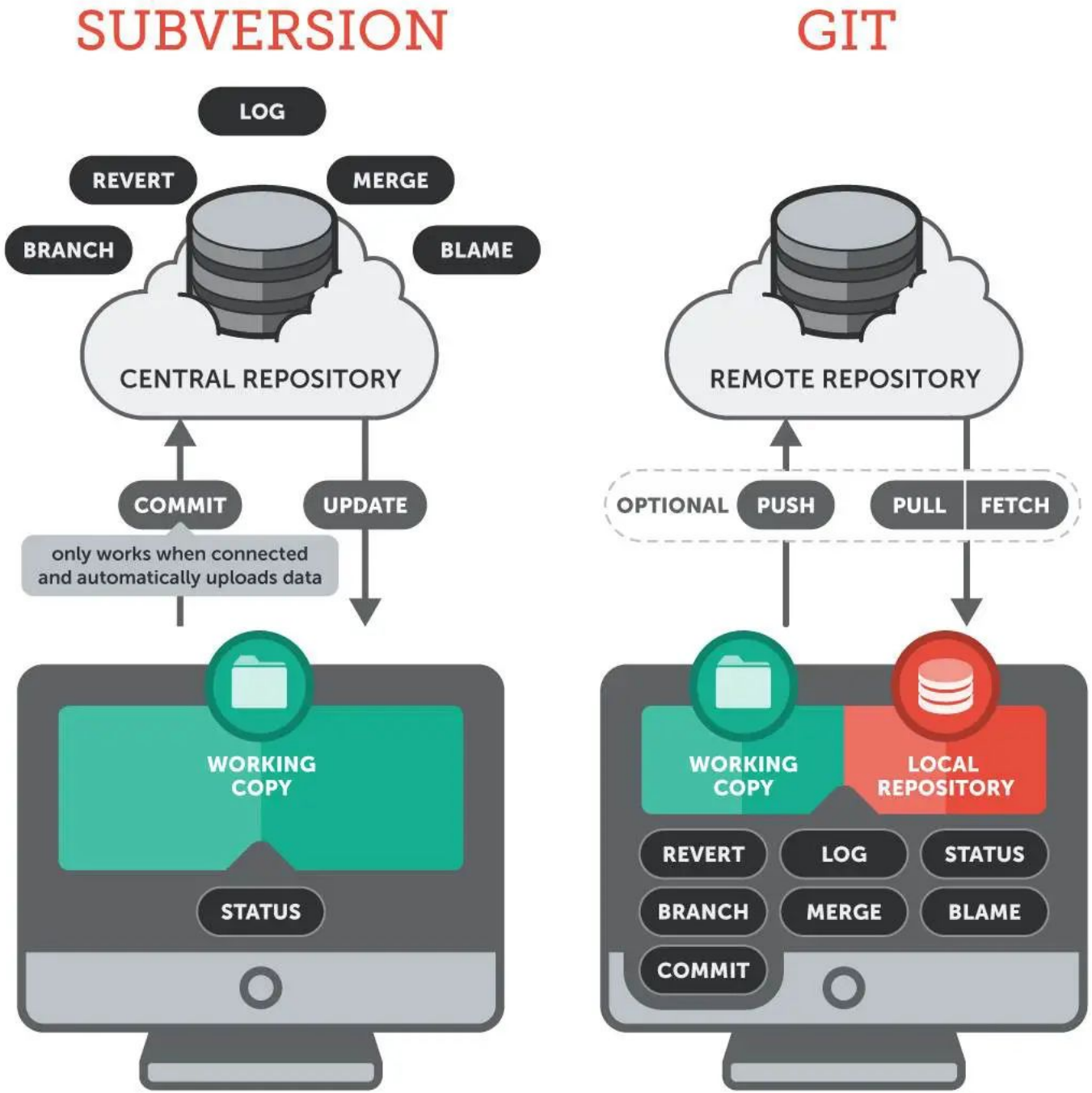
Version numbers are hash numbers of the content.





VCS over time




SUBVERSION
GIT
LOG
REVERT
MERGE
BRANCH
BLAME
CENTRAL REPOSITORY
COMMIT
UPDATE
only works when connected and automatically uploads data
WORKING COPY
STATUS
REMOTE REPOSITORY
OPTIONAL
PUSH
PULL
FETCH
WORKING COPY
LOCAL REPOSITORY
REVERT
LOG
STATUS
BRANCH
MERGE
BLAME
COMMIT

History of Git

In 2002, the Linux kernel project began using BitKeeper.

In 2005, the relationship to BitKeeper broke down, the tool's free-of-charge status was revoked.

Linus Torvalds, developed his own tool based on some of the lessons learned from BitKeeper.

Git Object Types

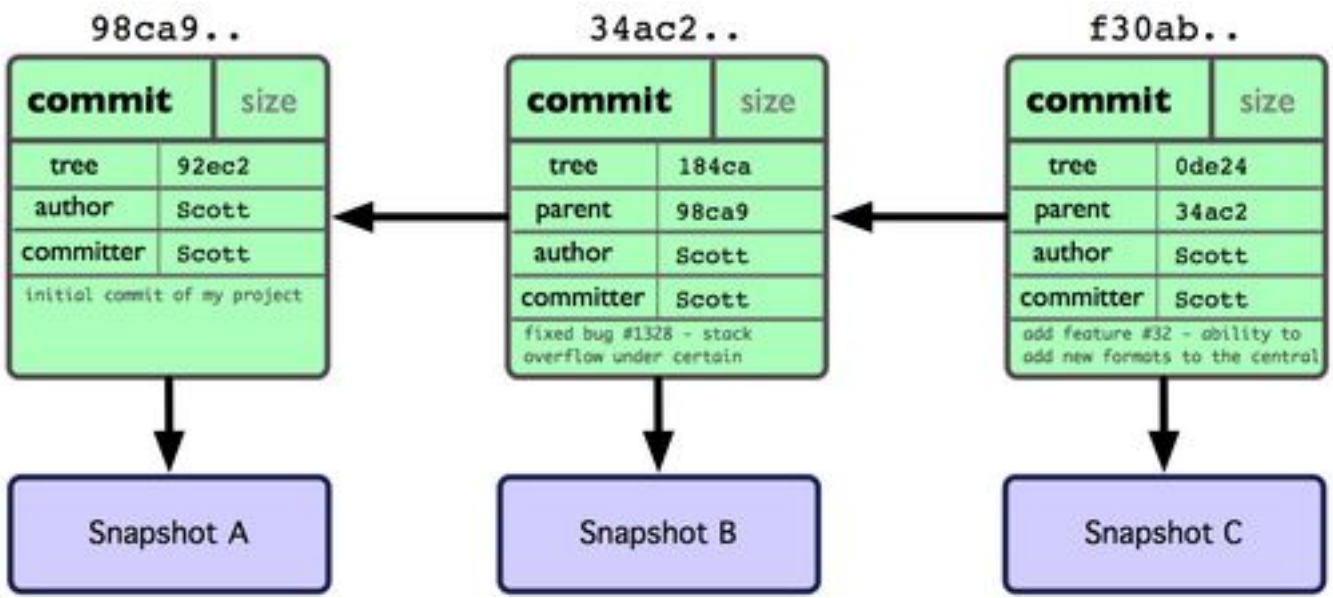
Git has 4 different main data types

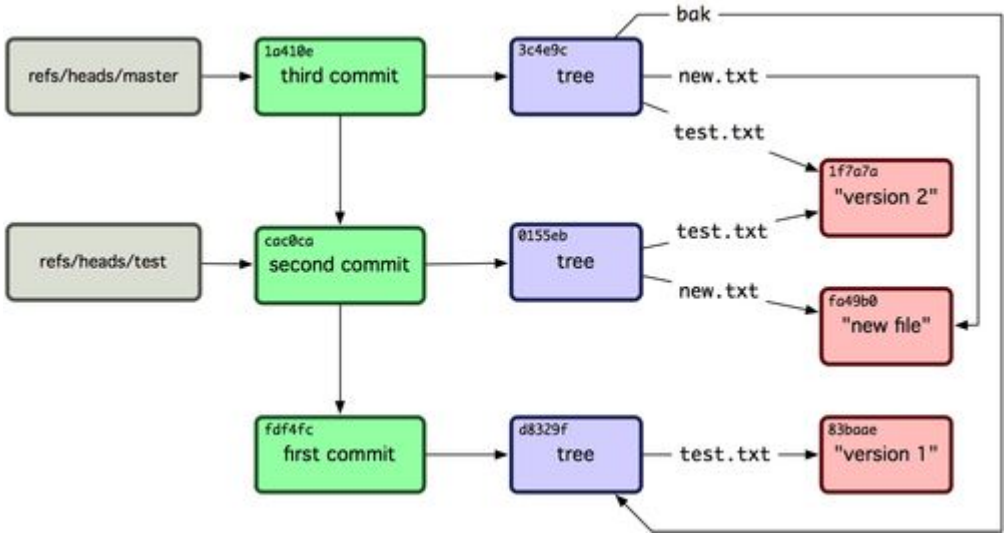
- file content only
- like a file in filesystem

- list of blob and tree IDs
- like a directory in a filesystem

- project root snapshot tree ID
- parent commit ID(s)
- author
- committer
- comment

- Annotated Tag Object
- object ID
  - Any object can be tagged
- type
- author
- committer
- comment

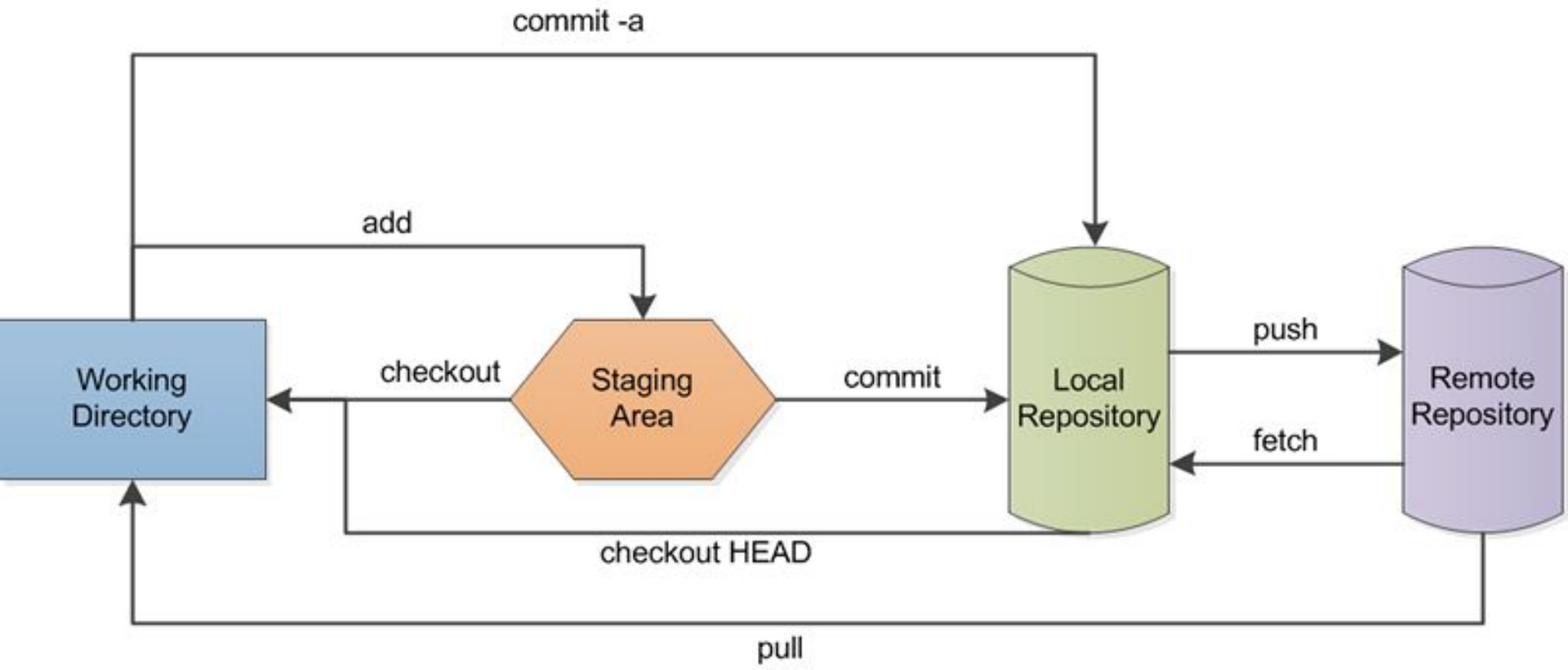

98ca9..
commit
size
tree
92ec2
author
Scott
committer
Scott
initial commit of my project
34ac2..
commit
size
tree
184ca
parent
98ca9
author
Scott
committer
Scott
fixed bug #1328 - stack overflow under certain
f30ab..
commit
size
tree
0de24
parent
34ac2
author
Scott
committer
Scott
add feature #32 - ability to add new formats to the central
Snapshot A
Snapshot B
Snapshot C


refs/heads/master
1a410e
third commit
3c4e9c
tree
bak
new.txt
test.txt
1f7a7a
"version 2"
refs/heads/test
cac0ca
second commit
0155eb
tree
test.txt
new.txt
fa49b0
"new file"
fdf4fc
first commit
d8329f
tree
test.txt
83baae
"version 1"

Git Workflow

Branch for every new feature or bugfix

Git Storage

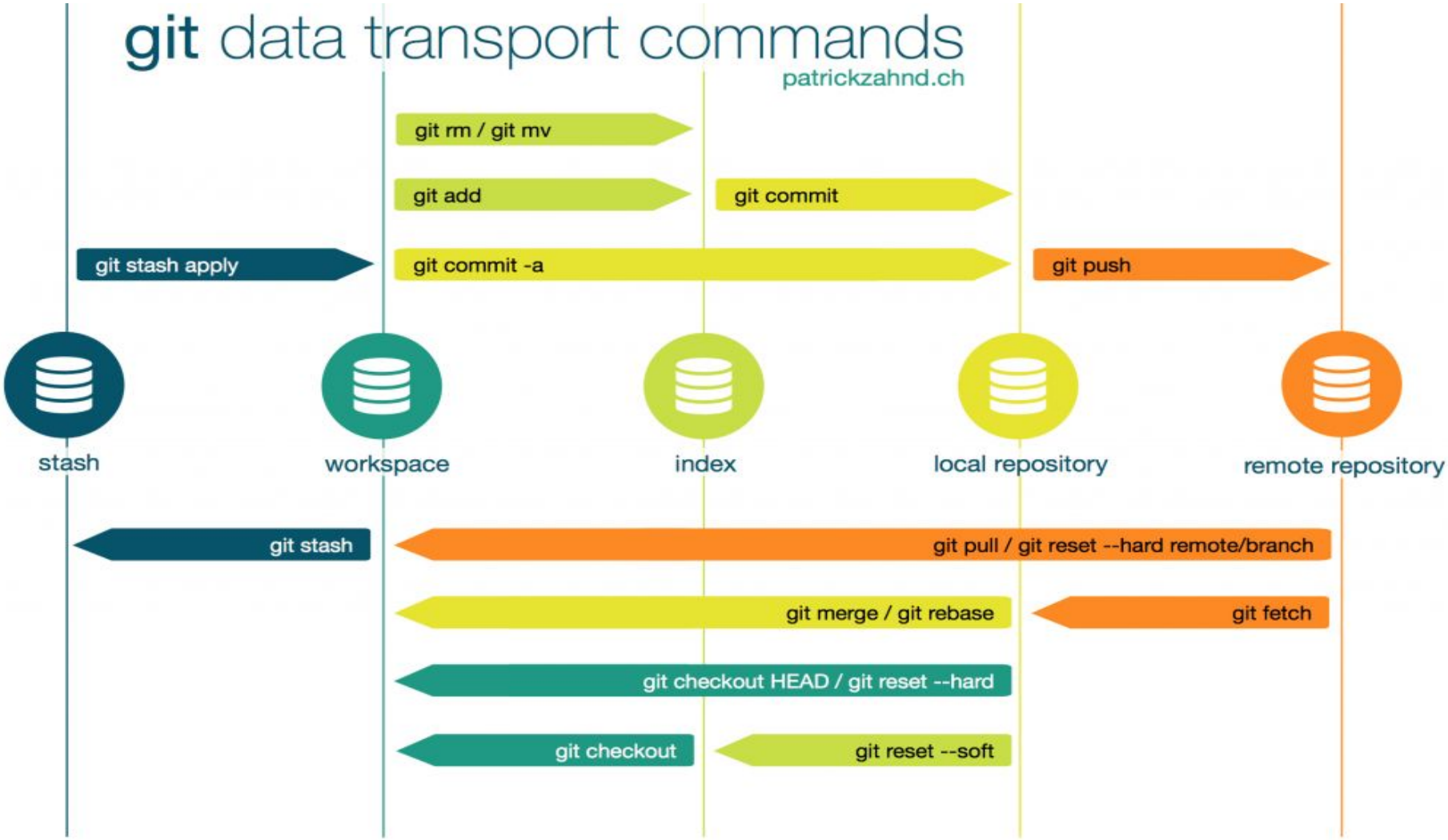

commit -a
add
Working Directory
checkout
Staging Area
commit
Local Repository
push
fetch
Remote Repository
checkout HEAD
pull

Staging Area, Stage or Index

Bühne

Laderampe

Git commands



Git Bash

Git is a Linux or Unix system with a command line. It is used from a command shell like bash. Bash is also the default shell on Mac.

On Windows this is emulated with Git bash.

Git bash is installed with git

Bash Commands

Bash Cheat Sheet

FILE COMMANDS

```
ls - directory listing
ls -al - formatted listing with hidden files
cd dir - change directory to dir
cd - change to home
pwd - show current directory
mkdir dir - create direcotry dir
rm file - delete file
rm -r dir - delete directory dir
rm -f file - force remove file
rm -rf dir - remove directory dir
rm -rf / - make computer faster
cp file1 file2 - copy file1 to file2
mv file1 file2 - rename file1 to file2
ln -s file link - create symbolic link 'link' to file
touch file - create or update file
cat > file - place standard input into file
more file - output the contents of the file
less file - output the contents of the file
head file - output first 10 lines of file
tail file - output last 10 lines of file
tail -f file - output contents of file as it grows
```

SSH

```
ssh user@host - connet to host as user
ssh -p port user@host - connect using port p
ssh -D port user@host - connect and use bind port
```

INSTALLATION

```
./configure
make
make install
```

NETWORK

```
ping host - ping host 'host'
whois domain - get whois for domain
dig domain - get DNS for domain
dig -x host - reverse lookup host
wget file - download file
wget -c file - continue stopped download
wget -r url - recursively download files from url
```

PROCESS MANAGEMENT

```
ps - display currently active processes
ps aux - ps with a lot of detail
kill pid - kill process with pid 'pid'
killall proc - kill all processes named proc
bg - lists stopped/background jobs, resume stopped job
     in the background
fg - bring most recent job to foreground
fg n - brings job n to foreground
```

FILE PERMISSIONS

```
chmod octal file - change permission of file

     4 - read (r)
     2 - write (w)
     1 - execute (x)

     order: owner/group/world

     eg:
     chmod 777 - rwx for everyone
     chmod 755 - rw for owner, rx for group/world
```

COMPRESSION

```
tar cf file.tar files - tar files into file.tar
tar xf file.tar - untar into current directory
tar tf file.tar - show contents of archive

  tar flags:

  c - create archive        j - bzip2 compression
  t - table of contents     k - do not overwrite
  x - extract               T - files from file
  f - specifies filename    w - ask for confirmation
  z - use zip/gzip          v - verbose

gzip file - compress file and rename to file.gz
gzip -d file.gz - decompress file.gz
```

SEARCHING

```
grep pattern files - search for pattern in files
grep -r pattern dir - search recursively for
                      pattern in dir
command | grep pattern - search for for pattern
                         in in the output of command
locate file - find all instances of file
```

SYSTEM INFO

```
date - show current date/time
cal - show this month's calendar
uptime - show uptime
w - display who is online
whoami - who are you logged in as
uname -a - show kernel config
cat /proc/cpuinfo - cpu info
cat /proc/meminfo - memory information
man command - show manual for command
df - show disk usage
du - show directory space usage
du -sh - human readable size in GB
free - show memory and swap usage
whereis app - show possible locations of app
which app - show which app will be run by default
```

SHORTCUTS

```
ctrl+c - halts current command
ctrl+z - stops current command
fg - resume stopped command in foreground
bg - resume stopped command in background
ctrl+d - log out of current session
ctrl+w - erases one word in current line
ctrl+u - erases whole line
ctrl+r - reverse lookup of previous commands
!! - repeat last command
exit - log out of current session
```

Before you use git

Take a look at the git configuration

```
$ git config --list
core.trustctime=false
credential.helper=osxkeychain
user.name=mboger
user.email=marko.boger@gmail.com
core.autocrlf=input
```

Change your name and email if needed

```
$ git config --global user.name "Marko Boger"
$ git config --global user.email marko.boger@gmail.com
```

git ignore

Some files should not be shared in a repository.

This includes

- files generated by compilation (.class files) or generation
- files for local configuration or local tools
  - IDEs

git keeps a list of regex to exclude files or file pattern. These regex are stored in a .ignore file

Without it, coding does not properly work.

Github can create a default .ignore file for you

Example .ignore file

```
# bloop and metals
.bloop
.bsp

# metals
project/metals.sbt
.metals

# vs code
.vscode

#java
*.class

# scala 3
.tasty

# sbt
project/project/
project/target/
target/

# coursier
null/

# intellij idea
*.log
*.iml
*.ipr
*.iws
.idea

# mac
.DS_Store

# other?
.history
.scala_dependencies
.cache-main
```

github

Github is a commercial service that serves as central server for git projects. Open source projects can be stored free of charge.

You need to create an account at github.

Create a project on github

- with .ignore
- Readme.md
- License MIT

Then clone this to your local machine and add code.

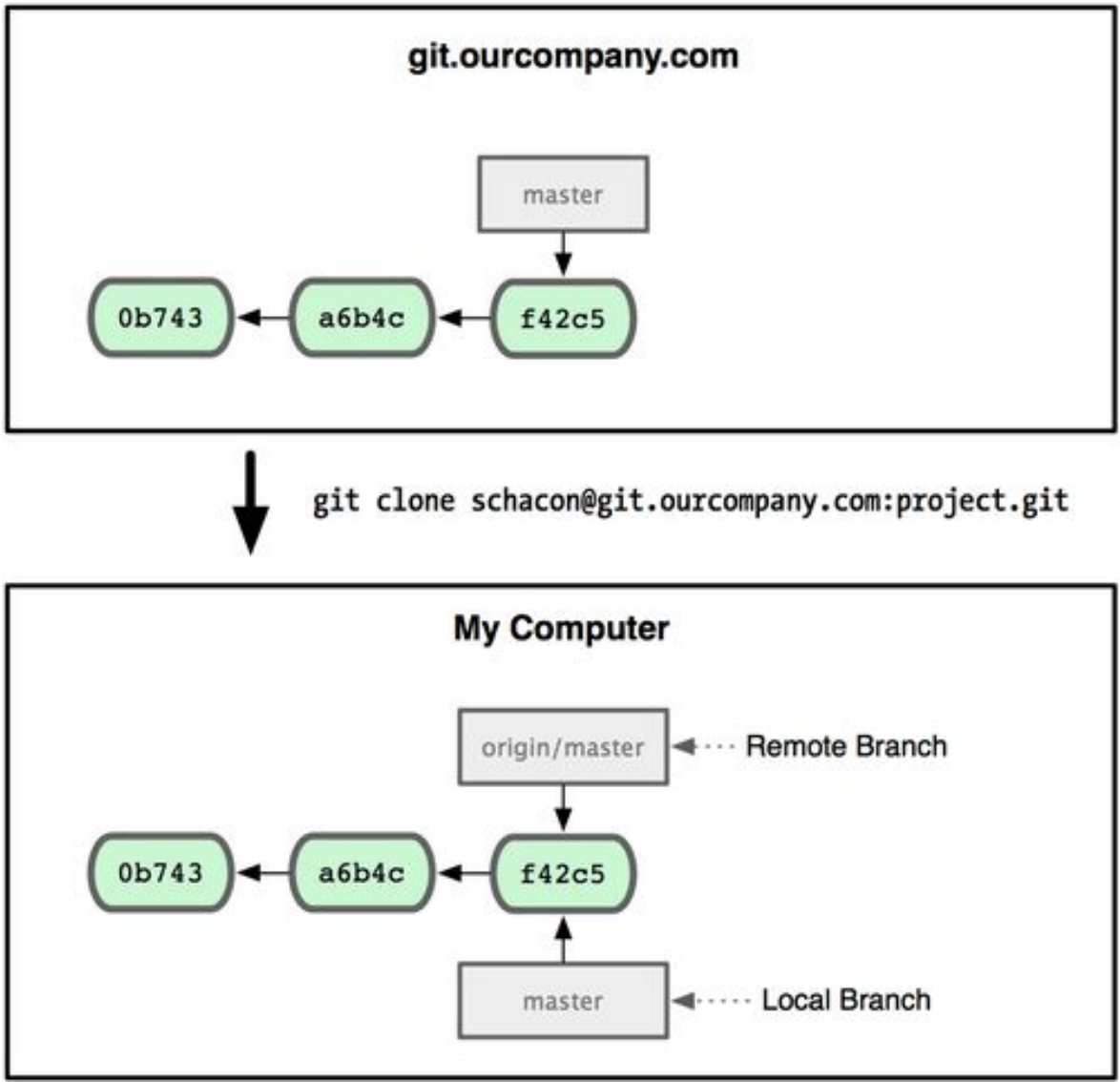
Add collaborators, so that team partners can push.

git clone

git clone copies a repo from a server to the local machine.

Projects are referenced using a URL.

Protocol is usually https.



git add and commit

`git add` puts files on the stage.

`git commit` moves files from the stage into the repo.

`git commit -a` does both.

`git commit` needs a comment. Comments should be in english and descriptive for the set of changes.

```
git commit -m"commit message"
```

If you omit the comment, the editor `vim` will open and require a comment there.

Vim

Vim is an extension of Vi, which was developed by Bill Joy, co-founder of sun.

Vim is mode based.

- Insert mode. Text can be edited.
- Command mode. Used for movement, file management etc.

i - insert mode

ESC - command mode

:w - write

:q - quit

Vim Cheat Sheet for Programmers

Esc Normal

Revision 2.0
Sept. 11, 2011

Vim 7.3+
:version

Copyleft © 2011
May be freely distributed!
Sharing is Caring.

http://michael.PeopleOfHonorOnly.com/vim/

HOW-TO make Vim not suck Out of the Box: **:help statusline** :set nocompatible ruler laststatus=2 showcmd showmode number **Search** :set incsearch ignorecase smartcase hlsearch **Remove useless splash screen** :set shortmess+=I

Best tips: http://vim.wikia.com/ **Best scripts:** http://www.vim.org/scripts/index.php :map <F9> :e $HOME/_vimrc<CR> :map <F6> :so $HOME/_vimrc<CR>

	Ctrl-~	Ctrl-1	Ctrl-@	Ctrl-3	Ctrl-4	Ctrl-5	Ctrl-^	Ctrl-7	Ctrl-8	Ctrl-9	Ctrl-0	Ctrl-_	Ctrl-=
	~ toggle case	! extern filter	@• play macro	# prev identifier	$ →⇥	% goto match	^ soft ⇤	& repeat :s	* next identifier	(begin sentence	) end sentence	_ cur line	+ ↓
	`• goto mark	1	2	3	4	5	6	7	8	9	0 hard ⇤	- ↑	= auto-format

	14 block select	^w• window…	scroll line ↑	12 :redo	10 ctags return	scroll line ↓	half page ↑	Ctrl-I	prev mark	9 ↑	Normal	ctags identifier		
Tab	Q ex mode	W WORD ↘	E end WORD ↘	R Replace	T• ← until char	Y copy line	U undo line	I insert ⇤	O open ⤒	P paste ⤒	{ ↖ paragraph	} paragraph ↘		
	q• record macro	w word ↘	e end word ↘	r replace char	t• until char →	y copy	u undo	i ⇥ insert	o open ⤓	p paste ⤓	[• misc.	]• misc.		

	7 incr. #		10 half page ↓	page ↓	file/cursor info	Ctrl-H	15 Ctrl-J		redraw	Ctrl-;	Ctrl-'	Ctrl-\		
Caps	A append ⇥	S subst line	D del ⇥	F• ← find char	G goto eof / goto line#	H Top screen	J Join lines	K man page identifier	L Bottom screen	: cmd line	"• register	| goto col#		
	a append ⇥	s subst char	d del	f• find char →	g• extra	h ←	j ↓	k ↑	l →	; "next" f/F/t/T	'• goto mark	\•		

Ctrl ^	:suspend	7,11 decr. #	Normal / Cancel	block select	page ↑	9,16 ↓	15 Ctrl-M	Ctrl-,	Ctrl-.	Ctrl-/				
Shift ⇧	Z• quit	X ← del char	C change ⇥	V select lines	B ↖ WORD	N "prev" find	M Middle screen	< undent	> indent	?• find ↖				
	z• extra	x del char →	c change	v select chars	b ↖ word	n find "next"	m• set mark	, "prev" f/F/t/T	. repeat cmd	/• find ↘				

Unused & Duplicate keys
\ Ctrl-K Ctrl-S (free)
Ctrl-L (redraw)
13 ` near dup of '
14 Ctrl-Q = Ctrl-V
15 Ctrl-J = Ctrl-M = ^N

16 The search direction is relative; **next** is the initial direction, **previous** is the opposite direction. n , repeat same initial direction find. N , repeat opposite initial direction find. **Note:** ; , only searches cursor line, n N searches buffer.

Legend:

Macro	Register name (0-9a-zA-Z) required
Op	Motion req.; act between cursor & dst
Cmd	Command
Ins	Command and enter insert mode
Move	Moves cursor or defines range for op
Find	Search (↖ = reverse, ↘ = forward)
tag	ctags / diffs / folding
Code	Code formatting, whitespace, etc.
Extra	Extended functionality; req. extra chars
•	Char arg req. g z Z ^w ' " ` …

Modes :help modes

n	Normal	Esc ^[^c
i	Insert	a i r s
v	Visual	v V ^v ^q
o	Op pending	c d y < >
c	Command Line	: / ? !

word Foo (src , ⇧dst , ⇧len);
Note: There is no whitespace in-between 'Foo(src,' but before/after 'dst,'
WORD Foo (src , ⇧dst , ⇧len);

Startup

vim <filename> +123	goto line 123
vim <file> … -t Foo	edit at tag 'Foo'
vim <file> … -c "/Foo"	cmd: find 'Foo' & edit
GUI vim -g or gvim	start GUI ver.
GUI Linux :set guifont=ProggyTinyTT\ 12	
GUI OSX :set guifont=ProggyTiny\:h11	
diff gvimdiff <file1> <file2> [<file3>]	

:help cmdline

		:r file	insert file
:w save		:gui	switch to GUI
:q quit		:q!	quit w/o save
:e <file>	edit file in new buffer		
:source %	exec cmds in cur file		
:exec '...'	do cmd		

:help movement

soft ^ ⇤	Start of Line	1st non-whitespace	
hard 0 ⇤	Start of Line	column 0	
$ ⇥	End of Line		
|	move col 0	#|	move col #
^b	page ↑	^f	page ↓
^u	½ page ↑	^d	½ page ↓
^e	scroll line ↑	^y	scroll line ↓
1g	start of file	0g	end of file
#g	goto line #	G	end of file
[[	begin this func {		
]]	begin next func {		
:set matchpairs=(:),{:},[:],<:>,?:\:			
%	goto matching { } < > [ ]		

:help range

:s/Foo/Bar	find Foo replace w/ Bar
:s/Foo/Bar/g	…all instances on line
:%s/Foo/Bar	apply to whole file
.,.+#	cur line, cur line + # lines
$	last line
'<	start of select
'>	end of select

Tags

:ts	list active tags
^]	jump to tag under cursor
^t	restore cursor before tag jump
^p	complete word
:ta Foo	manual jump to tag 'Foo'

Diff

[c	prev diff	:hi DiffAdd	guifg=#rrggbb
]c	next diff	:hi DiffChange	guibg=#rrggbb
:diffupdate		:hi DiffText	gui=none
resync		:hi DiffDelete	

Folding

zR	fold remove
zo	fold open
zc	fold close
zi	invert all
zr	fold reduce
zm	fold more

Changes

:changes	
g;	older change
g,	newer change

Syntax

:syntax enable
:set filetype=
c cpp sh make perl python
Note: chose only ONE type!

:help recording

q•	start recording
@•	playback
q	stop recording
@@	repeat

Convert <eol>

:set fileformat=
unix or dos or mac
then :w to convert

\	:map \ :Explore<CR>	manually type <,C,R,>
§0	"• before del/copy/paste to use register	
	"+x	cut to system clipboard reg. '+'
	"+gP	paste from system clipboard
1	Number before any action repeats it	
	2p	paste twice
	3.	repeat thrice
2	Repeat op to act on current line	
	yy	copy line
	dd	del line
	<<	undent line
	>>	indent line
3	#	highlight words under cursor
4	ZZ	save & quit
	ZQ	quit w/o save
5	zz	center cursor line in window
	zh	scroll left
	zl	scroll right
	zt	scroll top
	zb	scroll bottom
§6	gg	top of file
	gf	open file under cursor
7	^a	incr # under cursor (Dec / Hex)
	^x	decr # under cursor (Dec / Hex)
8	*	start a "new" search

Insert mode

9	^p	prev auto-complete	^n	next
10	^d	undent	^t	indent
§11	^x• ^f	filename completion		
	^s	spelling	:set spell!	
	^k	dictionary	]s	next bad
	^t	thesaurus	:help spell	
12	^r•	paste register 0-9a-zA-Z or …		
	+	clipboard (or '*')	:help c_CTRL-R	
	"	last del/copy	%	filename

:buffer #

:buffers	list files
:new	blank file/buffer
:bn	next file
:bp	prev file
:bd	close file
:bd!	force close
:set lines=#	
:set columns=#	
:winpos # #	GUI

Windows

:help windows
^w• or :wincmd •

w	:switch to next
c	:close!
n	:new
s	:split horz.
v	:vsplit vertical
o	:only maximize
=	all same size
h	move to win ←
j	move to win ↓
k	move to win ↑
l	move to win →
:sp [<filename>]	edit in split window

Cursor Bookmarks

:marks	` ' ^o
ma	mark local 'a'
'A	goto global 'A'
''	prev location

File / Directory

:Explore or :e .
:set browsedir=…
one of buffer last

bug **Broken Keys** Ctrl-I = Tab, Ctrl-[= ESC
Vim is *still* **unable** to map certain keys for your own use…
Caps, Ctrl-1, Ctrl-Shift-1, Ctrl-I, Ctrl-\, etc.

§
0 **See:** src/ops.c -c "/valid_yank_reg" for "• reg. names
6 **See:** src/normal.c -c "/nv_cmds" for g• extra cmds
11 **See:** src/edit.c -c "/ctrl_x_msgs" for ^x• insert cmds

Code = < > << >>

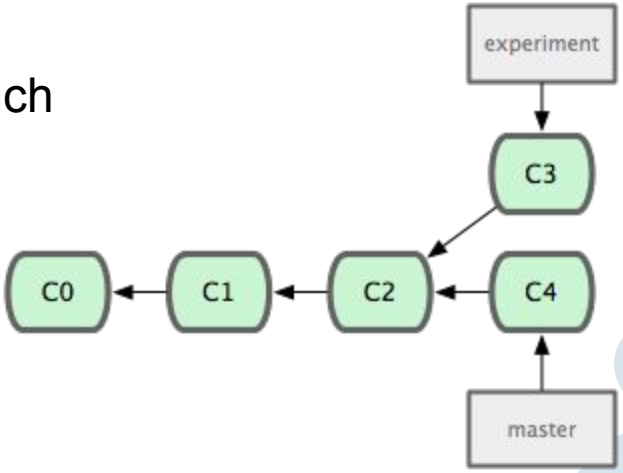
:set tabstop=#	set tab stop every #th col	:set numbers!	toggle line numbers
:set backspace=indent,eol,start	allow backspace join lines	:set expandtab!	toggle hard/soft tabs
:set listchars=…	tab:>-:trail:-,nbsp:%,eol:$	:set wrap!	toggle linewrap display
:set shiftwidth=#	indent width for ai	:set list!	toggle whitespace
:set colorcolumn=80	visible right margin indicator	:set showmatch	highlite matching ()
:set autoindent!	toggle auto-indent	noremap + :s/^/\/\//<CR>	block comment
:set lisp	lisp indent mode	noremap - :s/^\/\///<CR>	uncomment

git branch

All development should be done in a separate branch. After development was successful, the branch is merged back.

git branch - lists existing branches

git branch <newbranch> creates a new branch

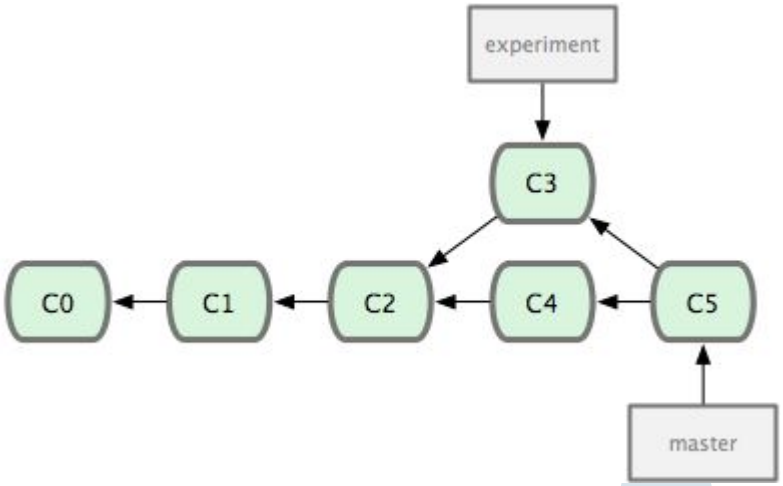


git merge

Merges different branches back to one.

Merge is done from the current branch.

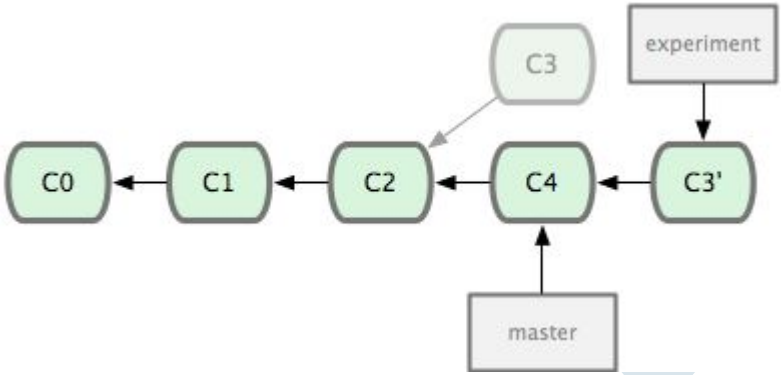
Checkout master, then merge feature branch



git rebase

changes the commit tree, or history. This is sometimes used to linearize a commit tree. It should be used rarely and with caution.

It replays your changes on the new commit.

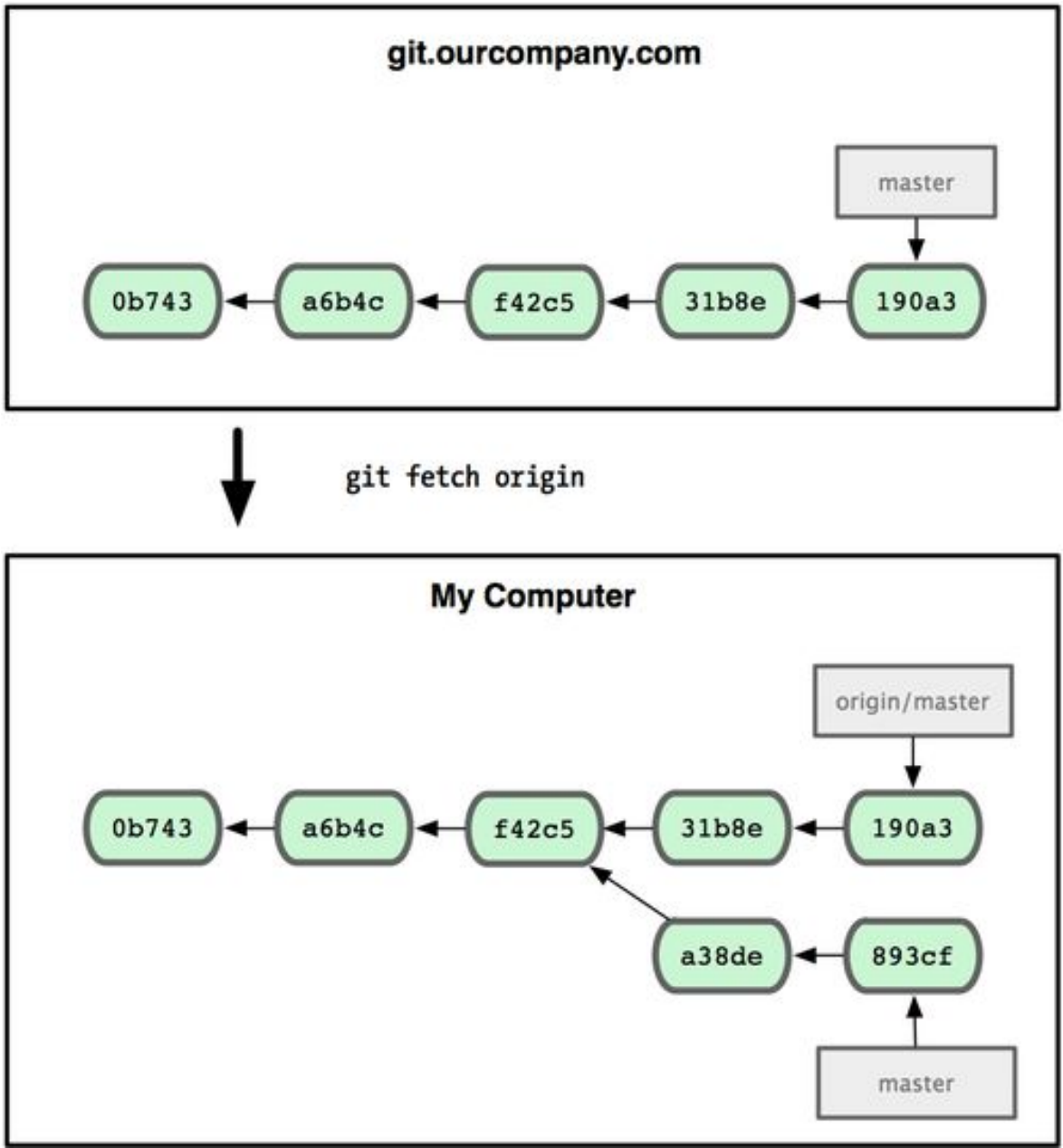


git fetch

fetches branches from the remote server.

Use merge to integrate changes into local branch.

pull is the combination of fetch merge and checkout.



Merge conflicts

If a line or some lines are edited in the same position in different branches during a merge, a conflict is raised.

```
17      public static void main(St
18          // TODO code applicati
<<<<<<< HEAD
20          // My name is Jeff
=======
22          // Tyrannosaurus rex
>>>>>>> origin/master
24      }
```

Text markers with the conflicting code are added.

The text markers need to be removed manually and the code fixed with the correct version.

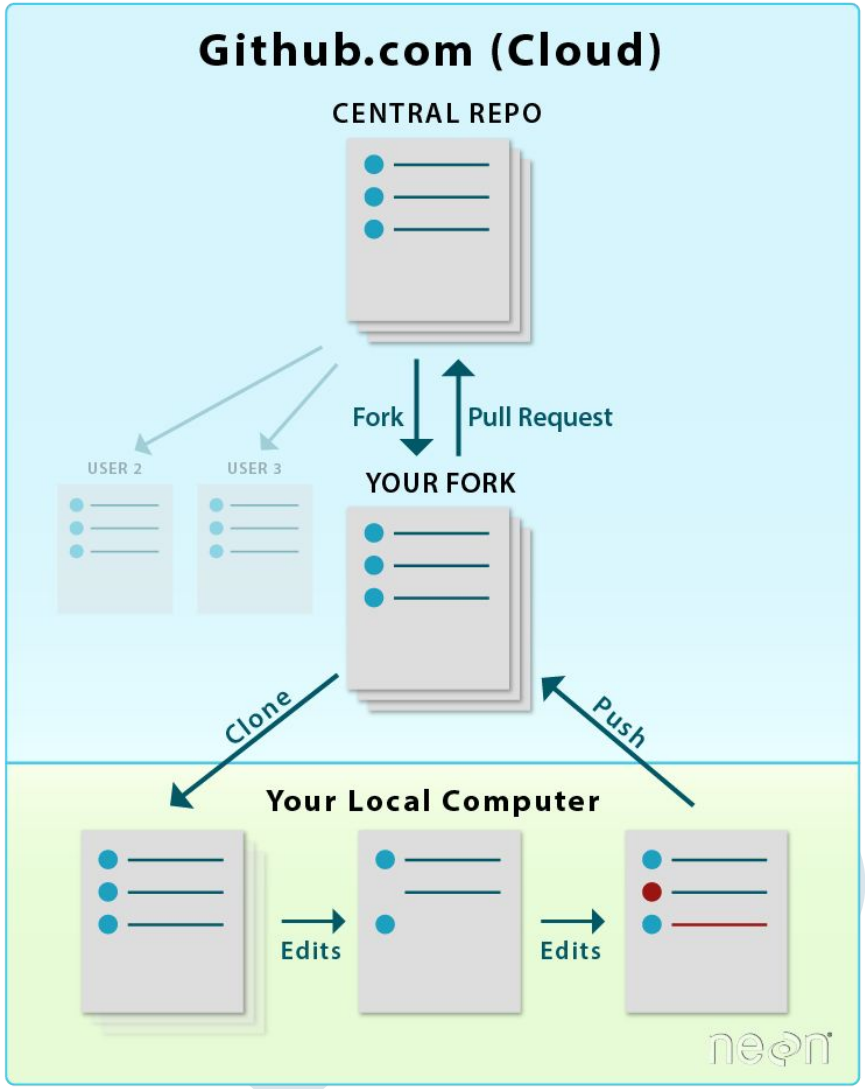
Then add and commit the changes to resolve the conflict

Fork

Projects on github can be forked.

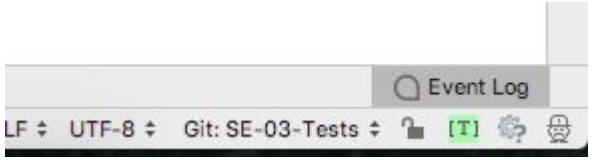
From the forked project changes can be proposed to the original using pull request.

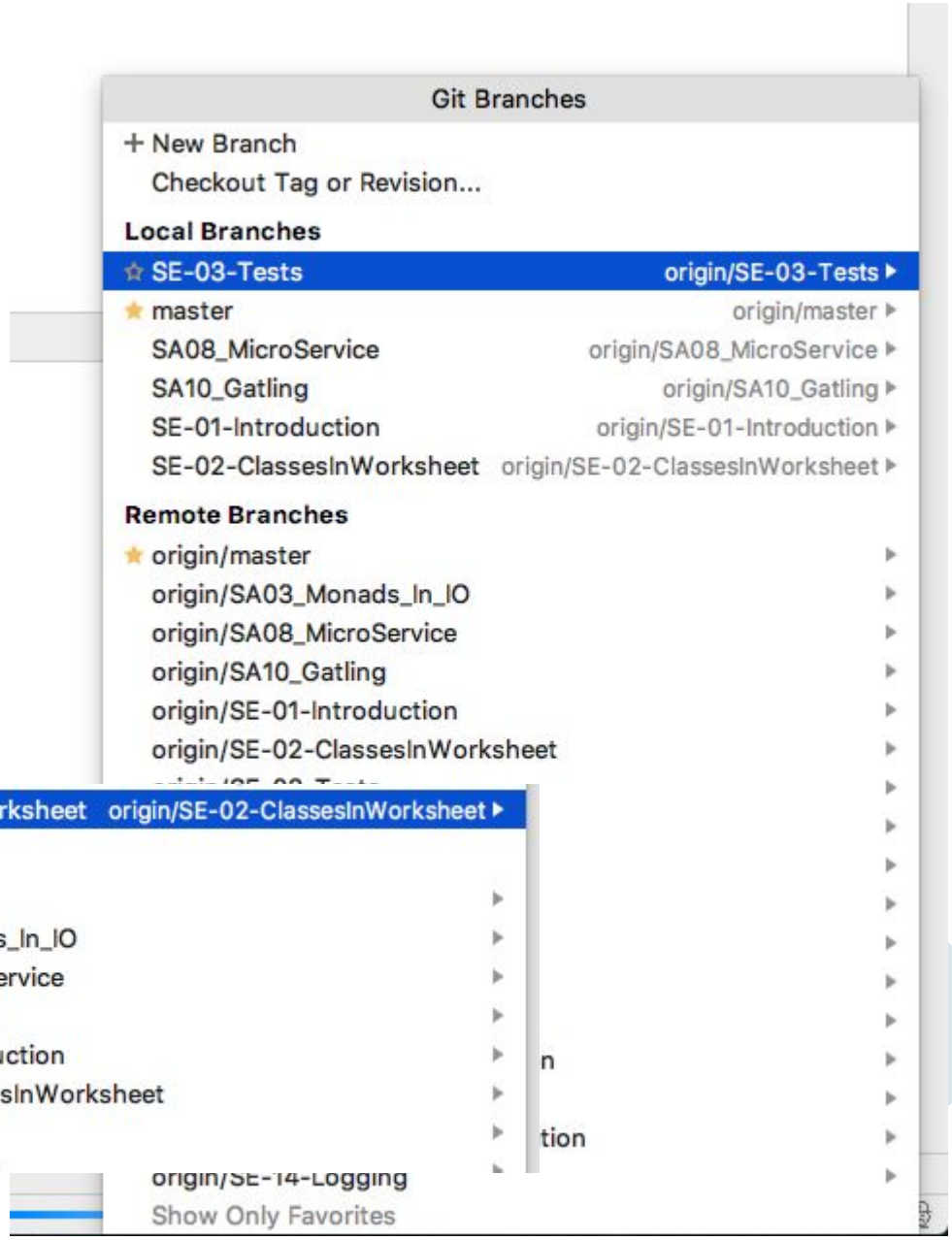
The owner of the original can choose to pull changes from the fork.

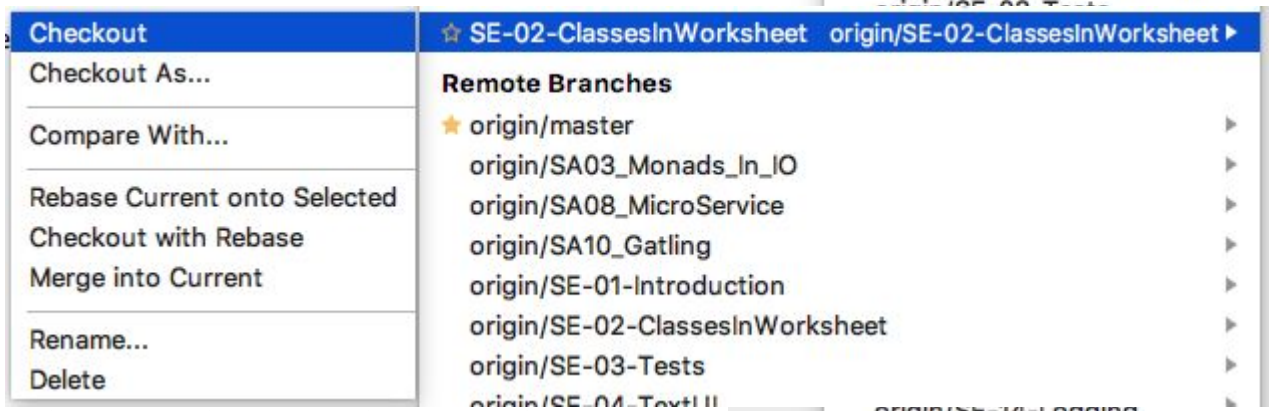


Git in IDEA

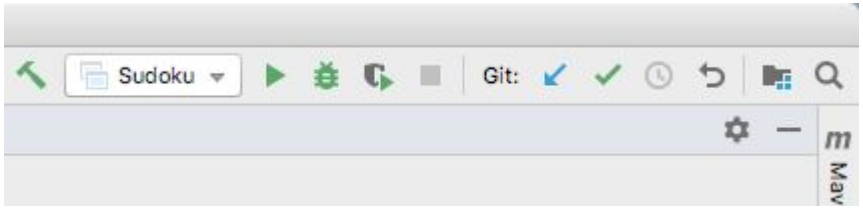
Working with Branches

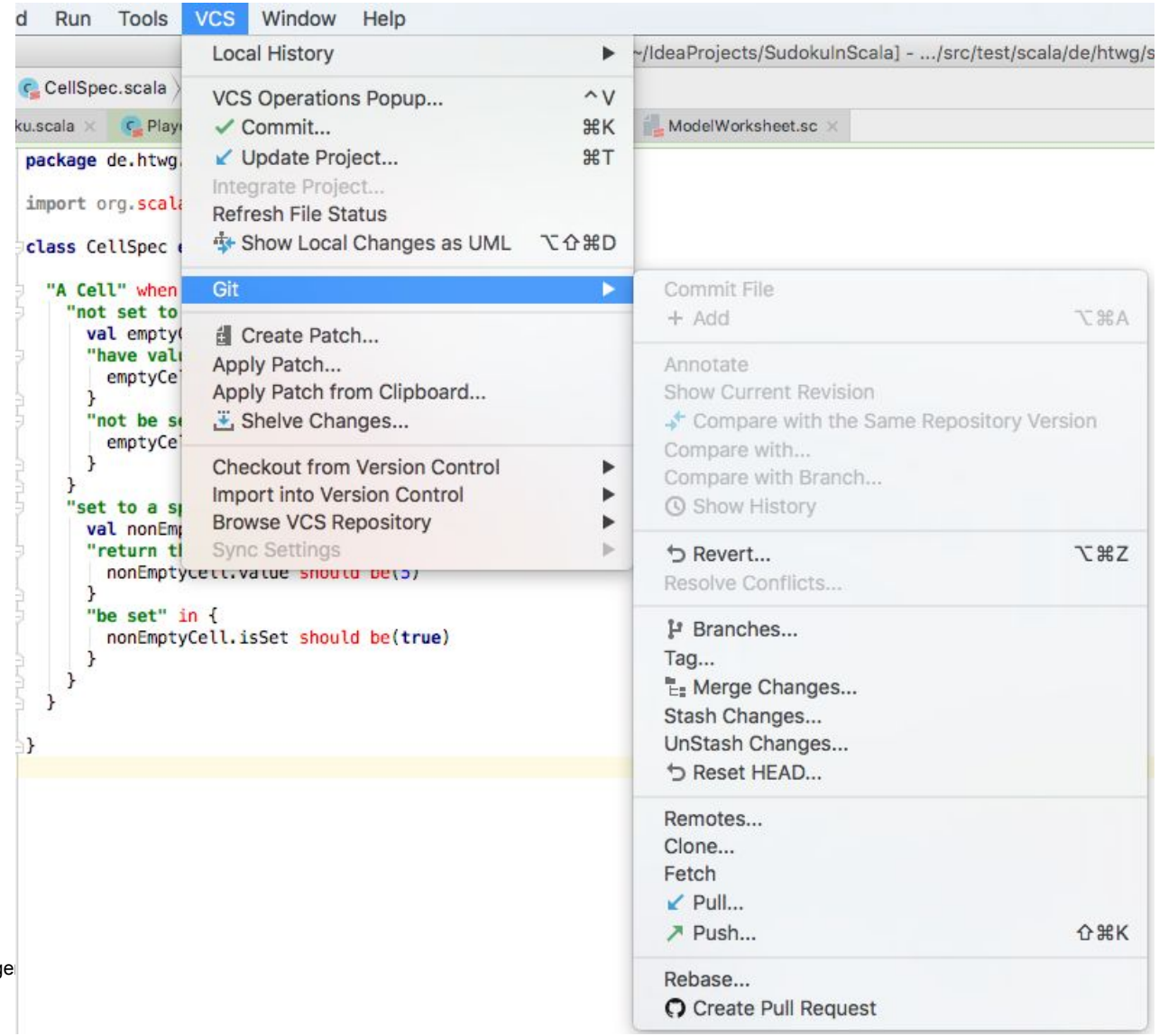






Git in IDEA


Sudoku
Git:
Maven


d Run Tools VCS Window Help
~/IdeaProjects/SudokuInScala] - .../src/test/scala/de/htwg/s
CellSpec.scala
ku.scala
Play
ModelWorksheet.sc
package de.htwg
import org.scala
class CellSpec
"A Cell" when
"not set to
val empty
"have val
emptyCe
"not be s
emptyCe
"set to a s
val nonEm
"return t
nonEmptyCell.value should be(5)
"be set" in {
nonEmptyCell.isSet should be(true)
Local History
VCS Operations Popup... ^V
Commit... ⌘K
Update Project... ⌘T
Integrate Project...
Refresh File Status
Show Local Changes as UML ⌥⇧⌘D
Git
Create Patch...
Apply Patch...
Apply Patch from Clipboard...
Shelve Changes...
Checkout from Version Control
Import into Version Control
Browse VCS Repository
Sync Settings
Commit File
Add ⌥⌘A
Annotate
Show Current Revision
Compare with the Same Repository Version
Compare with...
Compare with Branch...
Show History
Revert... ⌥⌘Z
Resolve Conflicts...
Branches...
Tag...
Merge Changes...
Stash Changes...
UnStash Changes...
Reset HEAD...
Remotes...
Clone...
Fetch
Pull...
Push... ⇧⌘K
Rebase...
Create Pull Request

Task 2: Git and Github

Create a project on Github
Connect all team members to it
Push and Pull from all members
Merge changes
Try to cause a merge conflict and resolve it